05 - AI Integration With Bedrock

In this exercise, you'll enhance your chatbot by integrating Amazon Bedrock foundation models. Instead of simple acknowledgment responses, your chatbot will provide intelligent, AI-generated replies using state-of-the-art language models.

What You'll Learn

By completing this exercise, you will:

- **Integrate** Amazon Bedrock foundation models into your Lambda function
- **Configure** IAM permissions for Bedrock access
- **Transform** your simple chatbot into an intelligent AI assistant
- **Experience** working with large language models through AWS services

Step 1: Add Bedrock Integration

Instruct Kiro to enhance your existing Lambda function with Bedrock capabilities:

```
Update my Lambda function to use Amazon Bedrock instead of returning simple ACK messages. Here are the requirements:

- Replace the current response logic with a call to Amazon Bedrock
- Use a foundation model like Claude or Titan to generate responses
- Send the user's message to the model and return the AI-generated response
- Update the IAM role to include necessary Bedrock permissions
- Keep the same API structure - don't change the frontend
- Handle errors gracefully if Bedrock is unavailable

Please update the CloudFormation template and Lambda code accordingly.
```

Backend Focus
This exercise only modifies the backend Lambda function. Your frontend will continue working exactly as before, but now it will receive intelligent AI responses instead of simple acknowledgments.

Step 2: Let Kiro Handle the Implementation

Allow Kiro to manage the technical implementation details:

```
Please handle the deployment for me. Update the CloudFormation stack with the new Lambda code and IAM permissions.
```

What Kiro Will Update

Your backend enhancement will include:

- **Lambda Function**: Updated code to call Bedrock APIs
- **IAM Permissions**: Added policies for Bedrock model access
- **Error Handling**: Graceful fallbacks if Bedrock calls fail
- **Response Format**: Maintains compatibility with your existing frontend

Seamless Integration
The beauty of this approach is that your frontend doesn't need any changes. The API contract remains the same, but now returns intelligent responses.

Step 3: Test Your AI Chatbot

Once deployed, please test the updated chatbot by sending a few different messages. Try questions like

```
What is AWS?
```

```
Tell me a joke
```

```
Explain cloud computing
```

Expected Behavior

Your chatbot should now be able to return AI generated responses.

▶ **Sample Conversation**

Step 4: Verify Everything Works

Confirm your AI integration is successful:

```
Please show me:
1. A successful test conversation with the AI
2. The CloudWatch logs showing Bedrock API calls
3. Confirmation that error handling works properly

This will verify the integration is complete and robust.
```

Success Criteria

After completing this exercise, you should have:

- ✅ **AI-Powered Responses**: Your chatbot now uses foundation models for intelligent replies
- ✅ **Bedrock Integration**: Lambda function successfully calls Amazon Bedrock APIs
- ✅ **Proper Permissions**: IAM roles configured for secure Bedrock access
- ✅ **Error Handling**: Graceful fallbacks when AI services are unavailable
- ✅ **Seamless Experience**: Frontend continues working without any changes

AI Chatbot Complete
Congratulations! You now have a fully functional AI-powered chatbot that can engage in intelligent conversations using Amazon Bedrock foundation models.

Troubleshooting

If you encounter issues, try these solutions:

▶ **Bedrock Access Denied**

▶ **Model Not Available**

▶ **Slow Responses**

Architecture Overview

▶ **View AI-Enhanced Architecture**

This exercise transforms your chatbot from a simple acknowledgment system into an intelligent AI assistant capable of natural language conversation.

AI Intelligence Components

Your enhanced Lambda function now includes:

- **Amazon Bedrock Integration**: Direct API calls to foundation models
- **Foundation Model Access**: Claude, Titan, or other state-of-the-art LLMs
- **Enhanced IAM Permissions**: Secure access to Bedrock services
- **Intelligent Response Generation**: Context-aware AI replies
- **Error Handling**: Graceful fallbacks when AI services are unavailable
- **Response Processing**: Formatting AI output for frontend consumption

Key AI Enhancement Benefits

- **Intelligent Conversations**: Natural language understanding and generation
- **Scalable AI**: Managed foundation models that scale automatically
- **Enterprise Security**: AWS-managed AI services with built-in security
- **Cost-Effective**: Pay-per-token pricing model
- **Multiple Models**: Access to various foundation models for different use cases
- **Seamless Integration**: No frontend changes required

Key Takeaways

This exercise demonstrated important AI integration concepts:

Technical Skills

- **AI Service Integration**: Connected your application to Amazon Bedrock
- **Permission Management**: Configured secure access to AI services
- **Error Handling**: Implemented robust fallback mechanisms
- **API Compatibility**: Maintained existing interfaces while adding new capabilities

AI Development Patterns

- **Seamless Enhancement**: Added AI without breaking existing functionality
- **Service Abstraction**: Used managed AI services instead of building from scratch
- **Scalable Architecture**: Leveraged AWS's global AI infrastructure
- **Cost-Effective**: Pay-per-use model for AI capabilities

Next Steps

Congratulations! Your chatbot now has AI intelligence and can engage in meaningful conversations. The next exercise will help you customize the user interface to create a polished, professional appearance.

Workshop Progress

Let's Customize Chatbot UI Design

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use Cookie preferences